

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для учёта комплектующих, сборки
и продажи персональных компьютеров»

Выполнил: Ануфриев Платон Дмитриевич

Группа: 241-327

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«_____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Ануфриеву Платону Дмитриевичу,

обучающемуся группы 241-327,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для учёта комплектующих, сборки и продажи персональных компьютеров»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Анализ предметной области учёта комплектующих, сборки и продажи персональных компьютеров	10	22.03.2026	
Задача 1.2. Обзор существующих веб-приложений и интернет-магазинов компьютерной техники	7	26.03.2026	
Задача 1.3. Анализ программных инструментов разработки веб-приложений	4	28.03.2026	
Задача 1.4. Формулировка цели, задач и требований к разрабатываемой системе	4	30.03.2026	
Раздел 2. Проектирование веб-приложения			

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.1. Анализ целевой аудитории и ролей пользователей	5	02.04.2026	администратор, менеджер, клиент
Задача 2.2. Проектирование функциональности приложения (use case, user story)	7	06.04.2026	
Задача 2.3. Проектирование модели данных (ER-диаграмма, физическая схема)	8	10.04.2026	не менее 3 сущностей
Задача 2.4. Разработка макетов основных страниц (wireframe)	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Реализация базовой структуры приложения и шаблонов страниц	5	21.04.2026	
Задача 3.2. Реализация аутентификации и проверки прав доступа по ролям	7	27.04.2026	
Задача 3.3. Реализация CRUD-интерфейса для пользователей, товаров, заказов и сборок ПК	7	03.05.2026	
Задача 3.4. Реализация модуля сборки ПК из комплектующих	10	09.05.2026	
Задача 3.5. Реализация агрегирования данных по остаткам, продажам и прибыли	6	15.05.2026	
Задача 3.6. Реализация загрузки и выгрузки файлов	5	19.05.2026	Excel/CSV
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.05.2026	
Задача 4.2. Тестирование и развёртывание приложения	4	24.05.2026	
Задача 4.3. Оформление пояснительной записки	3	26.05.2026	

Руководитель курсовой работы (проекта):

старший преподаватель кафедры «Инфокогнитивные технологии»

«___» _____ 2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания

«___» _____ 2026 г.

Дата сдачи выполненной работы

«___» _____ 2026 г.

Задание принял к исполнению

«____» _____ 2026 г.

(подпись)

П. Д. Ануфриев

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1 Анализ предметной области сборки и продажи ПК	8
1.2 Обзор существующих веб-приложений и интернет-магазинов .	9
1.3 Анализ программных инструментов разработки	11
1.4 Формулировка цели, задач и требований к системе	13
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	15
2.1 Анализ целевой аудитории и ролей пользователей	15
2.2 Описание функциональности приложения	17
2.3 Проектирование модели данных	21
2.4 Разработка макетов страниц	24
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	29
3.1 Реализация базовой структуры приложения и шаблонов страниц	29
3.2 Реализация аутентификации и проверки прав доступа	30
3.3 Реализация CRUD-интерфейсов	31
3.4 Реализация модуля сборки ПК из комплектующих (конфигуратор)	33
3.5 Реализация агрегирования данных по остаткам, продажам и прибыли	35
3.6 Реализация загрузки и выгрузки файлов	36
Выводы по главе 3	37
ГЛАВА 4. ОФОРМЛЕНИЕ ИТОГОВ РАБОТЫ	38
4.1 Организация Git-репозитория	38
4.2 Тестирование и развёртывание приложения	39
Выводы по главе 4	41
ЗАКЛЮЧЕНИЕ	42
ПРИЛОЖЕНИЕ А. ДИАГРАММА ВАРИАНТОВ ИСПОЛЬЗОВА- НИЯ	46

ВВЕДЕНИЕ

Актуальность темы. Рынок персональных компьютеров активно развивается, а спрос на индивидуальные сборки ПК стабильно сохраняется: пользователи заинтересованы в получении устройств, максимально соответствующих их требованиям по производительности, стоимости, внешнему виду и назначению. Это приводит к росту числа небольших компаний и частных специалистов, занимающихся продажей комплектующих, сборкой компьютеров под заказ и сервисным сопровождением. Для таких организаций критически важна автоматизация процессов учёта комплектующих, формирования сборок, оформления заказов и анализа результатов продаж: при отсутствии специализированной информационной системы возникают проблемы с контролем остатков на складе, отслеживанием движения товаров, ведением клиентских заказов и расчётом прибыли.

Степень разработанности проблемы. На рынке представлено множество интернет-магазинов компьютерной техники (DNS, «Регард», «НИКС», «Ситилинк») и универсальных CRM-/ERP-систем. Однако существующие решения либо ориентированы на крупный розничный сегмент и не учитывают специфику внутреннего учёта комплектующих при сборке ПК под заказ, либо избыточны и дороги для малого бизнеса. Это обуславливает целесообразность разработки специализированного веб-приложения, объединяющего функции каталога, склада, конфигуратора, заказов и аналитики в едином интерфейсе с разграничением прав доступа.

Объект исследования — процессы учёта комплектующих, сборки и продажи персональных компьютеров в малых компаниях.

Предмет исследования — веб-приложение, предназначенное для автоматизации учёта комплектующих, управления сборками ПК и оформления заказов с разграничением прав доступа по ролям.

Целью данной работы является проектирование и реализация веб-приложения, обеспечивающего автоматизацию учёта комплектующих, управление сборками ПК и взаимодействие администраторов, менеджеров и клиентов через систему личных кабинетов.

Для достижения цели необходимо решить следующие задачи:

- провести анализ предметной области и обзор существующих программных решений;

- обосновать выбор технологического стека для разработки;
- спроектировать архитектуру приложения, модель данных и пользовательские интерфейсы;
- реализовать аутентификацию, ролевую модель и CRUD-операции над основными сущностями;
- реализовать конфигуратор сборки, аналитические отчёты, импорт и экспорт данных;
- выполнить развёртывание приложения и оформить пояснительную записку.

Практическая значимость работы заключается в создании готового к внедрению программного продукта, который может использоваться действующими магазинами комплектующих и небольшими сборщиками ПК для автоматизации ключевых бизнес-процессов и сокращения операционных издержек.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ предметной области сборки и продажи ПК

Предметная область курсового проекта связана с деятельностью по закупке, хранению, учёту, сборке и продаже персональных компьютеров. Такая деятельность объединяет несколько взаимосвязанных процессов: работу со складом комплектующих, подбор совместимых компонентов, формирование готовых конфигураций, оформление заказов, взаимодействие с клиентами и получение аналитической информации по продажам.

Основой процесса является **складской учёт**. На складе могут находиться процессоры, материнские платы, видеокарты, оперативная память, накопители, блоки питания, корпуса, системы охлаждения и иные комплектующие. Для каждого товара необходимо хранить его наименование, категорию, количество, закупочную стоимость, продажную цену, характеристики и текущий статус наличия. При отсутствии автоматизированной системы ведение такого учёта вручную приводит к ошибкам, дублированию данных и трудностям при обновлении информации.

Следующий важный процесс — **сборка персонального компьютера**. В отличие от обычной продажи единичных товаров, сборка ПК требует учёта совместимости комплектующих, контроля остатков и фиксации состава каждой конфигурации. При формировании одной сборки используются сразу несколько единиц товаров из разных категорий. После подтверждения сборки количество соответствующих комплектующих на складе должно автоматически уменьшаться, что требует поддержки связи между складским учётом и сущностью сборки.

Ещё одним ключевым направлением является **работа с заказами**. Клиент может выбрать готовую конфигурацию, заказать сборку по своим параметрам либо приобрести отдельные комплектующие. Для каждого заказа требуется сохранять сведения о клиенте, составе заказа, дате оформления, сумме, статусе исполнения и способе оплаты. Кроме того, важно обеспечить возможность отслеживания истории заказов и формирования отчётных данных по продажам.

В рамках данной предметной области можно выделить несколько **категорий пользователей**. Администратор отвечает за полное управление системой, настройку справочников и управление доступом. Менеджер работает с това-

рами, заказами и клиентами, а также может создавать и редактировать сборки. Клиент взаимодействует с приложением как пользователь, который просматривает ассортимент, выбирает товары и оформляет заказ. Следовательно, в системе должна быть реализована ролевая модель доступа.

Особенностью данной предметной области является необходимость обработки **аналитической информации**: пользователю системы важно не только добавлять и изменять записи, но и получать агрегированные данные — общую сумму продаж за период, количество проданных товаров, наиболее востребованные комплектующие, остатки на складе, прибыль по заказам и по готовым сборкам.

Таким образом, предметная область включает совокупность процессов учёта товаров, формирования сборок, сопровождения заказов и анализа результатов продаж. Это делает разработку специализированного веб-приложения целесообразной и практически значимой.

1.2 Обзор существующих веб-приложений и интернет-магазинов

На современном рынке существует большое количество интернет-магазинов компьютерной техники и платформ для продажи комплектующих. К числу наиболее известных относятся крупные магазины, предлагающие широкий каталог товаров, фильтрацию по характеристикам, оформление заказов и базовые элементы личного кабинета пользователя. Такие решения ориентированы прежде всего на массовую торговлю и онлайн-продажи.

Несмотря на широкую распространённость подобных систем, они не всегда подходят для задач малого бизнеса или индивидуальной сборки ПК под заказ. Во многих случаях веб-сервисы ориентированы только на витрину товаров и не предоставляют полноценного механизма внутреннего учёта комплектующих, отслеживания фактических остатков или ведения истории использования компонентов в конкретных сборках.

Отдельно следует отметить **конфигураторы ПК**, позволяющие подобрать комплектующие по категориям. Их преимущество заключается в помощи пользователю при формировании конфигурации, однако подобные системы часто не интегрированы с внутренним складским учётом и механизмом управления заказами. В результате владелец бизнеса вынужден использовать сразу несколько разрозненных инструментов: отдельно таблицы для склада,

отдельно мессенджеры или CRM для клиентов и отдельно сайт для публикации товаров.

Также существуют **универсальные CRM- и ERP-системы**, которые могут использоваться для учёта товаров и заказов. Их достоинством является наличие модулей для управления клиентами и продажами. Однако такие решения часто являются избыточными по функциональности, сложны в настройке и адаптации под конкретный сценарий сборки персональных компьютеров, а их использование может быть экономически невыгодным для небольших проектов.

Для наглядного сопоставления существующих программных продуктов проведён сравнительный анализ четырёх характерных представителей рынка: крупного интернет-магазина электроники DNS, специализированной сети «Регард», магазина-конфигуратора «НИКС» и универсальной торговой площадки «Ситилинк». Сравнение по существенным для предметной области критериям приведено в таблице 1.1.

Таблица 1.1 – Сравнительный анализ существующих веб-приложений

Критерий	DNS	Регард	НИКС	Сити-линк	Разра-баты-ваемое
Каталог комплектующих	+	+	+	+	+
Конфигуратор сборки ПК	+	—	+	—	+
Контроль совместимости комплектующих	частично	—	+	—	+
Внутренний учёт остатков на складе	—	—	—	—	+
Ведение заказов в едином интерфейсе	+	+	+	+	+
Ролевая модель (ад-мин/менеджер/клиент)	—	—	—	—	+
Аналитика по продажам и прибыли	—	—	—	—	+
Импорт/экспорт каталога (CSV, Excel)	—	—	—	—	+
Адаптация под малый бизнес	низкая	низкая	сред-няя	низкая	полная
Стоимость использования для бизнеса	нет	нет	нет	нет	собств.

Результаты сравнения подтверждают сделанный ранее вывод: ни одно из рассмотренных решений не покрывает в комплексе задачи внутреннего учёта комплектующих, конфигурирования сборок, ведения заказов и аналитики. Существующие веб-приложения ориентированы прежде всего на витрину товаров для конечного покупателя и не предоставляют инструментов рабо-

ты для администратора и менеджера небольшой компании по сборке ПК. Это окончательно обосновывает целесообразность разработки собственного специализированного веб-приложения.

1.3 Анализ программных инструментов разработки

Для реализации веб-приложения необходимо выбрать стек технологий, который обеспечит сбалансированное соотношение скорости разработки, надёжности эксплуатации и удобства последующего сопровождения. В таблице 1.2 приведено сравнение основных программных инструментов, рассматривавшихся в качестве кандидатов для серверной части, клиентской части и хранилища данных.

На основании проведённого сравнения для реализации курсового проекта выбран следующий технологический стек:

- **серверная часть** — язык программирования Python 3.12 и фреймворк Django 5. Выбор обусловлен наличием встроенной административной панели, ORM для описания модели данных, готовых механизмов аутентификации, авторизации и валидации форм, а также большим объёмом учебной и справочной литературы [1];
- **клиентская часть** — HTML5, CSS3, CSS-фреймворк Bootstrap 5 и Vanilla JavaScript. Такой набор позволяет получить адаптивный интерфейс, не усложняя структуру проекта дополнительными сборщиками и SPA-фреймворками;
- **хранилище данных** — реляционная СУБД PostgreSQL 16 в качестве основной СУБД и SQLite на этапе локальной разработки. PostgreSQL обеспечивает поддержку ограничений целостности, транзакций и сложных аналитических запросов;
- **вспомогательные библиотеки** — Django REST framework для построения внутреннего API, библиотека `openpyxl` и стандартный модуль `csv` для импорта и экспорта данных, библиотека `Pillow` для работы с изображениями товаров;
- **инфраструктура и инструменты разработки** — система контроля версий Git и хостинг репозитория GitHub, среда разработки PyCharm

Таблица 1.2 – Сравнение программных инструментов разработки

Инструмент	Назначение	Преимущества	Недостатки
Django (Python)	Серверный фреймворк	Встроенная админ-панель, ORM, миграции, аутентификация, формы и валидация	Меньшая гибкость по сравнению с микрофреймворками
Flask (Python)	Серверный микрофреймворк	Минимализм, гибкость, простой запуск	Отсутствие готовой админки, много ручной работы
Node.js + Express	Серверная платформа	Высокая производительность, единый язык с фронтендом	Отсутствие готовой ORM, ручная сборка инфраструктуры
Laravel (PHP)	Серверный фреймворк	Зрелая экосистема, богатый набор пакетов	Менее распространён в российских учебных проектах
Bootstrap 5	CSS-фреймворк	Готовая адаптивная сетка, набор компонентов, быстрый старт	Типовой вид интерфейса без кастомизации
Vanilla JS	Клиентская логика	Отсутствие зависимостей, простота отладки	Ручная реализация повторяющихся компонентов
React / Vue.js	Клиентские фреймворки	Компонентный подход, удобство для SPA	Усложняют сборку и SEO в учебном проекте
PostgreSQL	СУБД	Сложные запросы, надёжная транзакционная модель, ограничения целостности	Требует более тщательной настройки
SQLite	СУБД	Не требует сервера, удобна для разработки	Ограниченная пропускная способность
Docker	Контейнеризация	Унификация окружения, простота деплоя	Дополнительный слой инфраструктуры
Git + GitHub	Контроль версий	История изменений, ревью кода, распределённая работа	Требует дисциплины ведения коммитов

Community / VS Code, контейнеризация средствами Docker и Docker Compose, обратный прокси-сервер Nginx и WSGI-сервер Gunicorn для последующего развёртывания приложения.

Выбранный стек удовлетворяет требованиям учебного проекта по дисциплине «Разработка веб-приложений», обеспечивает быстрый старт разработки за счёт готовых компонентов Django и Bootstrap и позволяет в дальнейшем масштабировать приложение без полной переработки исходного кода.

1.4 Формулировка цели, задач и требований к системе

На основе проведённого анализа предметной области, обзора существующих решений и анализа программных инструментов сформулированы требования к разрабатываемому веб-приложению.

Цель работы — разработка веб-приложения для учёта комплектующих, сборки и продажи персональных компьютеров с разграничением прав доступа администраторов, менеджеров и клиентов.

В системе должны быть реализованы функции **регистрации, аутентификации и авторизации** пользователей. Это необходимо для разграничения доступа к возможностям приложения в зависимости от роли пользователя. Администратор должен иметь полный доступ ко всем разделам системы, менеджер — доступ к управлению товарами, заказами и сборками, клиент — доступ к просмотру каталога, оформлению заказов и просмотру своих данных.

Важным требованием является наличие **CRUD-интерфейса** для основных сущностей приложения. Пользователь с соответствующими правами должен иметь возможность создавать, просматривать, изменять и удалять записи о товарах, сборках, заказах и пользователях.

Разрабатываемое приложение должно содержать **не менее трёх основных сущностей**. В рамках данного проекта такими сущностями являются:

- пользователи (с ролями);
- категории и комплектующие;
- сборки ПК и их состав;
- заказы и позиции заказов.

Кроме хранения и редактирования данных, система должна обеспечивать **обработку и агрегирование** информации: вычисление суммы продаж за период, определение количества товаров на складе, расчёт прибыли по заказам и отображение наиболее востребованных комплектующих.

Отдельным требованием является **поддержка загрузки и выгрузки файлов** в форматах CSV и Excel для импорта начального каталога, экспорта отчётных данных и резервного копирования.

К **нефункциональным требованиям** относятся удобство пользовательского интерфейса, надёжность хранения данных, безопасность (хранение

паролей в виде хэшей, защита от CSRF, XSS, SQL-инъекций), расширяемость и достаточная производительность при работе с каталогом, заказами и сборками.

Таким образом, разрабатываемая система должна представлять собой веб-приложение, объединяющее в себе механизмы учёта товаров, формирования сборок ПК, оформления заказов, разграничения доступа, загрузки и выгрузки файлов, а также аналитической обработки данных.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

Во второй главе курсовой работы выполняется проектирование разрабатываемого веб-приложения. На основании сформулированных в первой главе требований проводится анализ целевой аудитории и пользовательских ролей, формализуются функциональные и нефункциональные требования, описываются варианты использования и сценарии работы пользователей, проектируется модель данных приложения и разрабатываются макеты основных страниц.

2.1 Анализ целевой аудитории и ролей пользователей

Разрабатываемое веб-приложение ориентировано на небольшие компании и индивидуальных специалистов, занимающихся продажей комплектующих, сборкой персональных компьютеров под заказ и сопровождением заказов клиентов. Целевая аудитория системы делится на две принципиально разные группы: внутренние пользователи (сотрудники компании — администратор и менеджер) и внешние пользователи (клиенты — посетители сайта и зарегистрированные покупатели). Анализ предметной области показал, что в работе таких организаций задействованы три основные роли пользователей.

Администратор. Сотрудник, отвечающий за общее техническое сопровождение приложения, ведение справочников, управление учётными записями других пользователей и настройку параметров системы. Возрастной диапазон — 22–40 лет, уверенный пользователь компьютера, ежедневно работает с системой с настольного устройства. Ключевые ожидания: быстрый доступ к административной панели, возможность массового изменения справочников, чёткое разграничение прав менеджеров и клиентов.

Менеджер. Основной операционный пользователь системы. В обязанности менеджера входит ведение каталога комплектующих, оформление и сопровождение заказов клиентов, формирование сборок ПК, контроль остатков на складе и подготовка отчётности. Возрастной диапазон — 22–45 лет, средний и выше среднего уровень владения IT, работает с системой 6–8 часов в день преимущественно с настольного устройства. Ключевые ожидания: удобный CRUD-интерфейс для товаров, заказов и сборок; быстрый поиск по каталогу; наглядное отображение остатков; возможность импорта и экспорта данных.

Клиент. Конечный пользователь, для которого приложение является витриной товаров и средством оформления заказа. Возрастной диапазон — 18–45 лет, уровень владения IT — выше среднего. Использует приложение преимущественно с настольного устройства или планшета, частично — с мобильного телефона. Ключевые ожидания: удобный фильтр и поиск товаров по характеристикам, возможность собрать конфигурацию ПК из совместимых комплектующих, оформление заказа без длительной регистрации, просмотр истории заказов в личном кабинете.

Отдельной категорией являются **гости** — неавторизованные посетители сайта, имеющие доступ к публичному каталогу товаров и формам регистрации. Сводные характеристики ролей приведены в таблице 2.1.

Таблица 2.1 – Сравнительные характеристики ролей пользователей

Роль	Основные задачи	Частота использования	Устройство
Администратор	Управление пользователями и ролями, настройка справочников, аналитика	Несколько раз в неделю, 1–2 ч в день	Настольный ПК
Менеджер	Ведение каталога, оформление заказов, формирование сборок, контроль остатков	Ежедневно, 6–8 ч в день	Настольный ПК / ноутбук
Клиент	Просмотр каталога, конфигурирование сборки, оформление заказа	1–3 сессии при покупке	ПК, планшет, смартфон
Гость	Просмотр каталога, регистрация	Эпизодически	ПК, планшет, смартфон

На основе перечня ролей построена матрица прав доступа к основным сущностям системы (таблица 2.2). Матрица фиксирует разрешённые операции: С — создание, R — чтение, U — изменение, D — удаление.

Полученные сведения о целевой аудитории и матрице прав доступа учитываются при проектировании пользовательских интерфейсов: для администратора и менеджера предусматривается интерфейс с большим количеством таблиц, фильтров и действий, ориентированный на работу с настольного устройства; для клиента — более простой и адаптивный интерфейс с поддержкой мобильных устройств.

Таблица 2.2 – Матрица прав доступа (CRUD)

Сущность / действие	Гость	Клиент	Менеджер	Администратор
Каталог комплектующих	R	R	CRUD	CRUD
Готовые сборки ПК	R	R	CRUD	CRUD
Свой заказ	—	CRU	RU	RUD
Чужие заказы	—	—	RU	RUD
Свой профиль (клиент)	—	RU	R	RU
Учётные записи пользователей	—	—	—	CRUD
Справочники (категории, статусы)	—	—	R	CRUD
Аналитические отчёты	—	—	R	R

2.2 Описание функциональности приложения

На основе анализа предметной области, требований к системе и описанных пользовательских ролей сформирован перечень функциональных возможностей разрабатываемого веб-приложения. Функциональность представлена через перечень функциональных и нефункциональных требований, диаграмму вариантов использования, пользовательские истории и описание ключевых сценариев работы.

Действующими лицами (актёрами) системы являются: *гость* — неавторизованный посетитель сайта, *клиент* — зарегистрированный покупатель, *менеджер* — сотрудник, ведущий товары и заказы, *администратор* — сотрудник, имеющий полный доступ к системе.

Диаграмма вариантов использования (рисунок 2.1) объединяет в единой системе четыре актёра и пятнадцать прецедентов. Между ролями определены отношения наследования (клиент наследует возможности гостя, администратор — возможности менеджера), а между прецедентами «Оформление заказа» и «Регистрация и авторизация» определена связь *include*: оформление заказа неавторизованным пользователем возможно только после прохождения процедуры регистрации или входа.

Функциональные требования к системе систематизированы и приведены в таблице 2.3.

Нефункциональные требования, описывающие качества системы, приведены в таблице 2.4.

Функциональные требования декомпозированы на пользовательские



Рисунок 2.1 – Диаграмма вариантов использования (UML use case)

Таблица 2.3 – Функциональные требования к системе

ID	Требование	Описание
FR-1	Регистрация и аутентификация	Регистрация с email и паролем, авторизация, восстановление пароля
FR-2	Ролевая модель доступа	Назначение роли пользователю, проверка прав на каждом действии
FR-3	Управление каталогом	CRUD-операции над комплектующими, категориями и характеристиками
FR-4	Конфигуратор сборки ПК	Пошаговый выбор комплектующих с проверкой совместимости
FR-5	Управление сборками	CRUD-операции над готовыми сборками с указанием состава и цены
FR-6	Корзина и оформление заказа	Добавление товаров и сборок в корзину, оформление заказа
FR-7	Управление заказами	Просмотр и изменение статусов заказов менеджером, фильтрация
FR-8	Личный кабинет клиента	История заказов, текущие заказы, профиль, изменение пароля
FR-9	Импорт и экспорт данных	Импорт каталога из CSV/Excel с валидацией, экспорт в Excel
FR-10	Аналитические отчёты	Отчёты по выручке, прибыли, остаткам, популярным позициям
FR-11	Уведомления	Email о смене статуса заказа, низких остатках и новых заказах
FR-12	Управление пользователями	Создание учётных записей сотрудников, изменение ролей, блокировка

Таблица 2.4 – Нефункциональные требования к системе

ID	Категория	Требование
NFR-1	Производительность	Время отклика страниц каталога не более 2 с при 50 одновременных пользователей
NFR-2	Масштабируемость	Возможность размещения базы и приложения на отдельных серверах
NFR-3	Безопасность	Хранение паролей в виде хэшей, защита от CSRF, XSS, SQL-инъекций
NFR-4	Надёжность	Резервное копирование БД ежедневно, журналирование действий
NFR-5	Удобство использования	Адаптивная вёрстка (mobile first), освоение интерфейса менеджером за 1 ч
NFR-6	Совместимость	Поддержка актуальных версий браузеров Chrome, Firefox, Safari, Edge
NFR-7	Сопровождаемость	Соответствие стилю PEP 8, покрытие тестами не менее 60 % бизнес-логики
NFR-8	Расширяемость	Возможность добавления новых ролей и сущностей без переработки ядра

истории по ролям [2]. Истории для роли «Клиент» приведены в таблице 2.5, для роли «Менеджер» — в таблице 2.6, для роли «Администратор» — в таблице 2.7.

Таблица 2.5 – Пользовательские истории для роли «Клиент»

№	User story	Критерий приёмки
US-1	Как клиент, я хочу искать комплектующие по категории и характеристикам	В каталоге доступны фильтры по категории, цене и характеристикам
US-2	Как клиент, я хочу собрать конфигурацию ПК из совместимых комплектующих	В конфигураторе несовместимые позиции скрываются
US-3	Как клиент, я хочу оформить заказ из корзины	Поддерживается оформление с указанием контактов и оплаты
US-4	Как клиент, я хочу видеть историю своих заказов	В личном кабинете список заказов с датой, составом, суммой и статусом
US-5	Как клиент, я хочу видеть наличие товара на складе	Карточка товара отображает фактическое количество в наличии
US-6	Как клиент, я хочу получать уведомления о смене статуса	При смене статуса отправляется email

Описание ключевых сценариев. Для иллюстрации работы системы рассмотрим два ключевых сценария.

Сценарий 1. Оформление заказа сборки клиентом. Клиент открывает страницу конфигуратора, последовательно выбирает процессор, материнскую плату, оперативную память, накопитель, видеокарту, блок питания и корпус.

Таблица 2.6 – Пользовательские истории для роли «Менеджер»

№	User story	Критерий приёмки
US-7	Как менеджер, я хочу добавлять и редактировать комплектующие	Доступна форма создания и редактирования товара
US-8	Как менеджер, я хочу формировать готовые сборки ПК	При сохранении сборки комплектующие резервируются
US-9	Как менеджер, я хочу менять статусы заказов клиентов	В списке заказов поддерживается изменение статуса
US-10	Как менеджер, я хочу импортировать каталог из Excel/CSV	Имеется страница импорта с предпросмотром и валидацией
US-11	Как менеджер, я хочу выгружать отчёт по продажам в Excel	Кнопка «Экспорт» формирует файл за выбранный период
US-12	Как менеджер, я хочу видеть товары с низким остатком	На рабочем столе виджет «Низкие остатки» (порог 3 шт.)

Таблица 2.7 – Пользовательские истории для роли «Администратор»

№	User story	Критерий приёмки
US-13	Как админ, я хочу создавать учётные записи менеджеров и назначать роли	Страница списка пользователей с действиями «Создать», «Изменить роль»
US-14	Как админ, я хочу видеть выручку и прибыль за период	Дашборд с графиками и фильтром по датам
US-15	Как админ, я хочу управлять справочниками	Доступны CRUD-страницы для каждого справочника
US-16	Как админ, я хочу видеть журнал действий пользователей	Страница журнала с фильтрацией по пользователю и дате

После каждого шага система фильтрует доступные позиции по совместимости и обновляет итоговую цену. По завершении выбора клиент добавляет сборку в корзину, открывает корзину и переходит к оформлению заказа. Система запрашивает контактные данные, способ оплаты и доставки, сохраняет заказ в статусе «Новый», уменьшает остатки выбранных комплектующих и отправляет уведомление менеджеру.

Сценарий 2. Обработка заказа менеджером. Менеджер на рабочем столе видит уведомление о новом заказе. Открыв заказ, проверяет состав сборки, фактическое наличие комплектующих и при необходимости связывается с клиентом. После сборки ПК менеджер изменяет статус заказа на «Готов к выдаче» — система отправляет клиенту email с инструкциями. После передачи заказа менеджер устанавливает статус «Выполнен», фиксируя дату завершения.

2.3 Проектирование модели данных

Модель данных приложения разрабатывается с учётом сформулированных функциональных и нефункциональных требований. На логическом уровне выделено восемь основных сущностей: *Пользователь*, *Категория комплектующих*, *Комплектующее*, *Сборка ПК*, *Состав сборки*, *Заказ*, *Позиция заказа* и *Статус заказа* [3].

ER-диаграмма приложения (рисунок 2.2) построена в нотации Crow's foot и содержит описанные сущности с атрибутами и связями. Каждая сущность имеет первичный ключ id и системные атрибуты created_at / updated_at. Между сущностями определены связи «один ко многим» (например, *Категория* – *Комплектующее*) и «многие ко многим», реализованные через ассоциативные таблицы build_items и order_items. Связи имеют ограничения целостности (ON DELETE CASCADE для дочерних записей сборки и заказа, ON DELETE RESTRICT для пользователей с активными заказами).

В таблице 2.8 приведён перечень основных таблиц физической схемы базы данных в формате PostgreSQL, в таблицах 2.9–2.11 — словари данных для трёх ключевых сущностей.

Таблица 2.8 – Основные таблицы базы данных

Таблица	Назначение	Ключевые поля
users	Учётная запись пользователя	id, email, password_hash, role, full_name, phone, is_active, created_at
categories	Справочник категорий комплектующих	id, title, slug, description
components	Комплектующие (товары)	id, category_id, title, brand, model, specs (JSON), purchase_price, sale_price, stock, image
builds	Готовая или клиентская сборка ПК	id, title, description, total_price, created_by, created_at
build_items	Состав сборки	id, build_id, component_id, quantity
orders	Заказ клиента	id, client_id, build_id, total_sum, status, payment_method, created_at
order_items	Позиции заказа	id, order_id, component_id, quantity, unit_price
order_statuses	Справочник статусов заказа	id, code, title, color

Между сущностями устанавливаются следующие связи:

- *Категория* — *Комплектующее*: один ко многим;

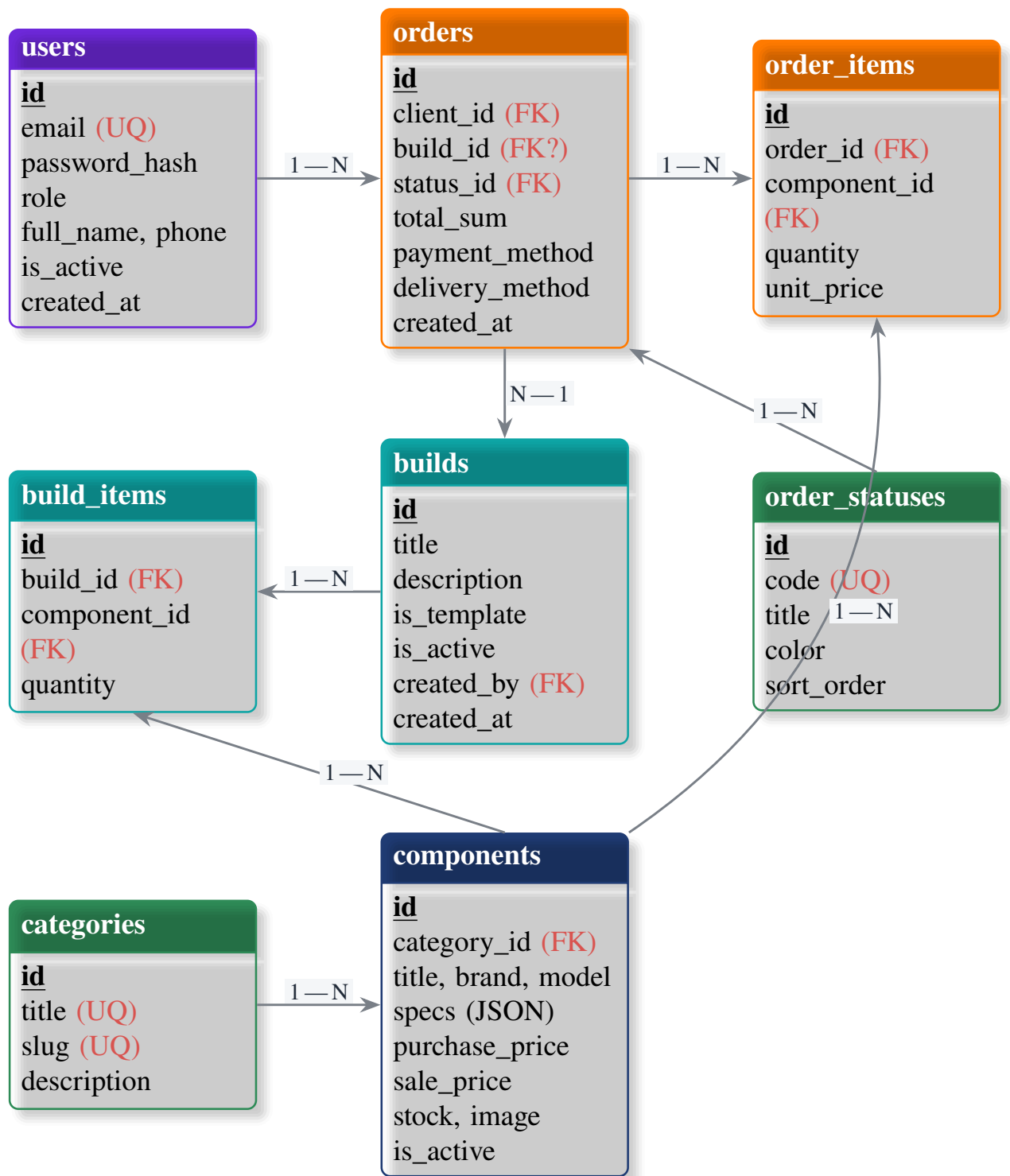


Рисунок 2.2 – ER-диаграмма базы данных (нотация Crow's foot)

- Сборка — Состав сборки — Комплектующие: «многие ко многим» через таблицу build_items;
- Пользователь (клиент) — Заказ: один ко многим;
- Заказ — Позиция заказа — Комплектующие: «многие ко многим» через таблицу order_items;

Таблица 2.9 – Словарь данных таблицы users

Поле	Тип	Назначение и ограничения
id	SERIAL PK	Первичный ключ
email	VARCHAR(254) UQ NOT NULL	Логин пользователя, индекс
password_hash	VARCHAR(128) NOT NULL	Хэш пароля (PBKDF2)
role	VARCHAR(16) NOT NULL	client / manager / admin
full_name	VARCHAR(150)	Полное имя пользователя
phone	VARCHAR(20)	Контактный телефон
is_active	BOOLEAN DEFAULT TRUE	Активна ли учётная запись
created_at	TIMESTAMP NOT NULL	Дата создания записи

Таблица 2.10 – Словарь данных таблицы components

Поле	Тип	Назначение и ограничения
id	SERIAL PK	Первичный ключ
category_id	INT FK	Ссылка на categories.id
title	VARCHAR(200) NOT NULL	Наименование товара
brand	VARCHAR(100)	Производитель
model	VARCHAR(100)	Модель
specs	JSONB	Технические характеристики
purchase_price	NUMERIC(10,2)	Закупочная цена
sale_price	NUMERIC(10,2) NOT NULL	Продажная цена, не менее 0
stock	INT NOT NULL DEFAULT 0	Текущий остаток на складе
image	VARCHAR(255)	Путь к изображению товара

- *Заказ — Сборка*: необязательная связь «многие к одному»;
- *Заказ — Статус заказа*: «многие к одному» через справочник order_statuses.

Индексы и оптимизация запросов. Для обеспечения быстрого поиска и фильтрации в каталоге товаров предусмотрены индексы по полям:

- components.category_id и components.brand — для фильтров каталога;
- GIN-индекс по полю components.specs (JSONB) — для поиска по техническим характеристикам;

Таблица 2.11 – Словарь данных таблицы orders

Поле	Тип	Назначение и ограничения
id	SERIAL PK	Первичный ключ
client_id	INT FK NOT NULL	Ссылка на users.id
build_id	INT FK NULL	Опциональная ссылка на сборку
total_sum	NUMERIC(12,2) NOT NULL	Итоговая сумма заказа
status	VARCHAR(16) NOT NULL	Код статуса заказа
payment_method	VARCHAR(20) NOT NULL	Способ оплаты
delivery_method	VARCHAR(20) NOT NULL	Способ получения
delivery_address	VARCHAR(255)	Адрес доставки
created_at	TIMESTAMP NOT NULL	Дата оформления заказа
updated_at	TIMESTAMP NOT NULL	Дата последнего обновления

- `orders.client_id` и `orders.created_at` — для отчётности и личного кабинета;
- уникальный индекс по полю `users.email` — для быстрой авторизации и предотвращения дублирования учётных записей.

Физическая реализация осуществляется средствами миграций Django, что обеспечивает воспроизводимость структуры на любой среде разработки и развёртывания.

2.4 Разработка макетов страниц

На основе сформулированных функциональных требований, пользовательских сценариев и спроектированной модели данных разработаны макеты основных страниц веб-приложения. Прототипы выполняются в инструменте Figma. Основные страницы сгруппированы по ролям пользователей и связаны между собой картой переходов (таблица 2.12).

Главная страница (рисунок 2.3) состоит из «шапки» с логотипом и ссылками на каталог и configurator, «героя» с описанием магазина и кнопкой «Собрать ПК», блока популярных сборок, блока акций и «подвала» со ссылками на разделы.

Таблица 2.12 – Карта сайта по ролям пользователей

Раздел	Доступ	Состав страниц
Публичная часть	Все	Главная, каталог, карточка товара, конфигуратор, регистрация, авторизация
Личный кабинет клиента	Клиент, Админ	Дашборд, активные заказы, история, профиль, корзина
Рабочее место менеджера	Менеджер, Админ	Рабочий стол, каталог, сборки, заказы, импорт/экспорт, отчёты
Панель администратора	Админ	Пользователи, роли, справочники, аналитика, журнал

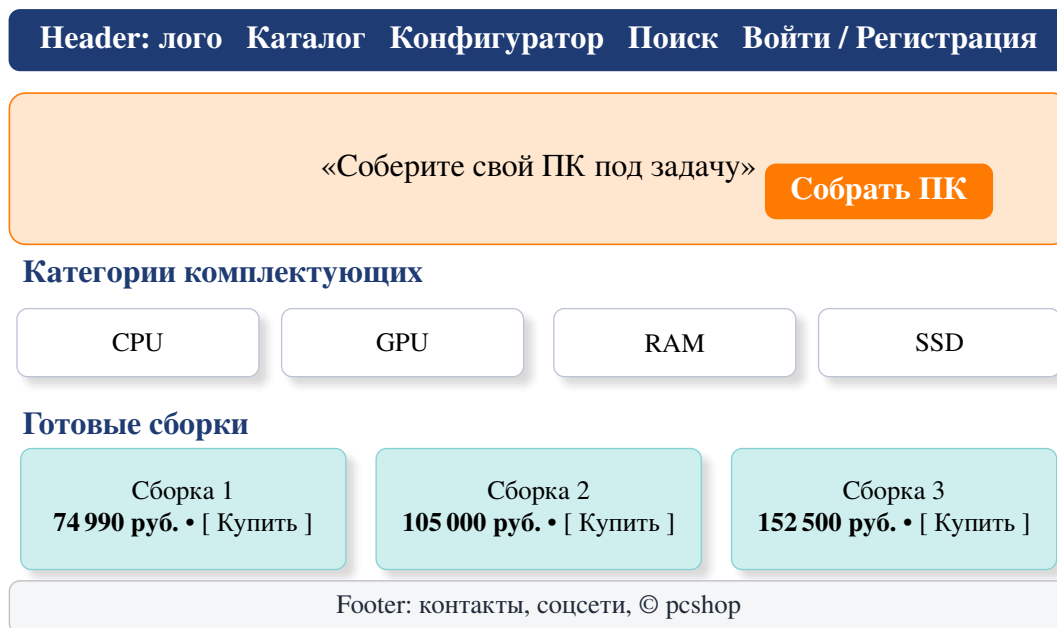


Рисунок 2.3 – Wireframe главной страницы

Каталог товаров (рисунок 2.4) содержит панель фильтров слева (категория, цена, бренд, характеристики, остаток) и сетку карточек с пагинацией. Каждая карточка показывает изображение, наименование, бренд, цену, остаток и кнопку «Добавить в корзину».

Конфигуратор сборки (рисунок 2.5) реализован как пошаговый мастер с подсветкой активного шага. На каждом шаге отображается панель выбора одной категории комплектующих, под ней — сводка текущей сборки и итоговая сумма.

Корзина и оформление заказа (рисунок 2.6) объединяет список добавленных товаров и сборок с возможностью изменения количества и удаления. Оформление выполняется одной страницей с секциями «Контактные данные», «Способ оплаты», «Способ получения» и итоговым блоком с кнопкой «Оформить заказ».

Личный кабинет клиента (рисунок 2.7) включает виджеты «Актив-

Header / Хлебные крошки: Главная → Каталог → Процессоры

Фильтры

☐ Категория
 ☐ Бренд

Цена, руб.
 [____] — [____]

☐ В наличии
 ☐ Низкий остаток

[Применить]

Карточка товара 1

цена руб.
[В корзину]

Карточка товара 2

цена руб.
[В корзину]

Карточка товара 3

цена руб.
[В корзину]

Карточка товара 4

цена руб.
[В корзину]

Карточка товара 5

цена руб.
[В корзину]

Карточка товара 6

цена руб.
[В корзину]

Пагинация: 1 2 3 ... N

Рисунок 2.4 – Wireframe страницы каталога товаров

Конфигуратор сборки ПК

1. CPU

2. MB

3. RAM

4. SSD

5. GPU

6. PSU

7. Корпус

Шаг 2. Материнская плата

Фильтры: форм-фактор, чипсет, тип памяти, Wi-Fi.
 [Карточка платы 1] [Карточка 2] [Карточка 3]
 [Карточка платы 4] [Карточка 5] [Карточка 6]
 [Выбрать]

Текущая сборка

CPU: AMD 7700X ✓
 MB: *выбирается*
 RAM: ____
 SSD: ____
 GPU: ____
 Итого: 27 990 руб.

[Назад]

Сохранить сборку

Далее

Оформить заказ

Рисунок 2.5 – Wireframe конфигуратора сборки ПК

Корзина и оформление заказа

Товары в корзине

1. Ryzen 7 7700X — 27 990 руб. [− 1 +] [удалить]
 2. Сборка «Игровая средняя» — 110 000 руб. [удалить]

Контактные данные и доставка

Email: _____
 Телефон: _____
 Оплата: ● Картой ○ СБП ○ Наличные
 Получение: ● Самовывоз ○ Курьер

Итого

Товары: 137 990 руб.
 Доставка: 0 руб.
 Сумма: 137 990 руб.
 [Оформить заказ]

Рисунок 2.6 – Wireframe корзины и оформления заказа

ные заказы» и «Сохранённые сборки», подробные блоки «История заказов» (таблица с фильтрами) и «Адреса доставки».



Рисунок 2.7 – Wireframe личного кабинета клиента

Рабочее место менеджера (рисунок 2.8) включает виджеты «Новые заказы», «В работе», «Низкие остатки», а также таблицу заказов с поиском и фильтрами. Через основное меню доступны страницы каталога, сборок, импорта/экспорта и отчётов.



Рисунок 2.8 – Wireframe рабочего места менеджера

Панель администратора (рисунок 2.9) включает разделы «Пользователи», «Справочники», «Аналитика» и «Журнал действий». Основной экран аналитики содержит графики выручки и прибыли, таблицу топ-товаров и фильтры по периоду.

При разработке макетов соблюдаются принципы: адаптивный дизайн (mobile first) на основе сетки Bootstrap 5; минимизация числа действий (оформление заказа за три перехода); единый визуальный стиль (общая цветовая палитра, типографика Roboto, набор иконок Bootstrap Icons); соблюдение

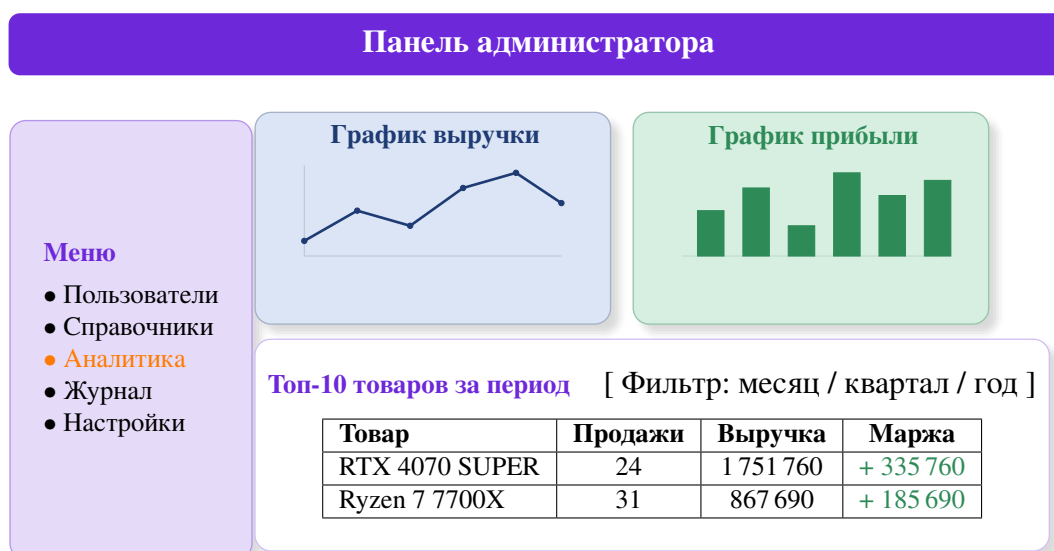


Рисунок 2.9 – Wireframe панели администратора

требований доступности WCAG 2.1 AA; обратная связь пользователю (всплывающие уведомления, индикаторы загрузки, валидация форм). Основной цвет интерфейса — глубокий синий (#1F3B73), цвет акцента — оранжевый (#FF7A00), нейтральный фон — светло-серый (#F4F6F8).

Полученные результаты проектирования — ролевая модель, функциональные требования, модель данных и макеты основных страниц — являются исходной базой для последующей программной реализации, описанной в третьей главе пояснительной записки.

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

В третьей главе описывается программная реализация веб-приложения. Кодовая база организована как Django-проект `pcshop`, разделённый на четыре приложения (Django apps): `accounts` (пользователи, аутентификация и аналитика), `catalog` (категории и комплектующие), `builds` (сборки ПК, состав и конфигуратор), `orders` (корзина, заказы и статусы). Реализация выполнялась на основе официальной документации Django [4] и учебных материалов [1].

3.1 Реализация базовой структуры приложения и шаблонов страниц

Базовая структура проекта построена в соответствии с принципом «приложение на смысловую область»: каждая Django-app содержит модели, миграции, представления, формы, маршруты и админ-регистрацию своей сущности. Глобальные настройки (`settings.py`), маршрутизация (`urls.py`) и WSGI-точки входа собраны в каталоге `pcshop/`. В качестве СУБД по умолчанию используется SQLite (без дополнительной настройки), переключение на PostgreSQL выполняется через переменные окружения в файле `.env` с помощью пакета `python-decouple`.

Структура каталогов проекта приведена в таблице 3.1.

Таблица 3.1 – Структура каталогов проекта `pcshop`

Каталог / файл	Назначение
<code>pcshop/</code>	Корневой пакет проекта: <code>settings.py</code> , <code>urls.py</code> , <code>views.py</code> , <code>wsgi.py</code>
<code>accounts/</code>	Пользователь, авторизация, ролевые миксины, аналитика, дашборд
<code>catalog/</code>	Категории, комплектующие, импорт/экспорт CSV
<code>builds/</code>	Сборки ПК, состав, конфигуратор
<code>orders/</code>	Корзина, заказы, позиции, статусы, контекст-процессор корзины
<code>templates/</code>	HTML-шаблоны Bootstrap 5, сгруппированные по приложениям
<code>fixtures/</code>	Стартовые данные (статусы заказов, категории, образцы товаров)

Клиентская часть реализована на CSS-фреймворке Bootstrap 5 (подключается через CDN). Разработан единый базовый шаблон `templates/base.html`, обеспечивающий:

- стек технологий Google Fonts (Inter для текста, JetBrains Mono для кода);
- «стеклянную» липкую навигационную панель с эффектом `backdrop-filter: blur` и адаптивным меню по ролям пользователя;
- единую палитру (основной цвет — индиго #4F46E5, акцент — янтарный #F59E0B), скругления, мягкие тени, `hover`-эффекты;
- современную типографику и адаптивную сетку для мобильных устройств.

Главная страница `templates/home.html` содержит hero-блок с градиентным фоном и призывом к действию, сетку категорий с иконками Bootstrap Icons, блок популярных сборок и блок преимуществ. Все вторичные шаблоны (каталог, корзина, личный кабинет, аналитика и др.) наследуются от базового через `{% extends %}`.

Для отображения числа позиций корзины в навигационной панели реализован контекст-процессор `orders/context_processors.py`, зарегистрированный в `TEMPLATES[...] ['OPTIONS'] ['context_processors']`. Процессор читает данные только из сессии браузера (без обращений к базе данных) и добавляет переменную `cart_count` в контекст каждого шаблона. На иконке «Корзина» отображается оранжевый числовой бейдж при наличии позиций.

3.2 Реализация аутентификации и проверки прав доступа

Для разграничения доступа реализована собственная модель пользователя `accounts.User`, наследующаяся от `AbstractUser`. Логин по умолчанию заменён на `email`: установлены `USERNAME_FIELD = 'email'` и `REQUIRED_FIELDS = []`, поле `username` отключено. Модель содержит поле `role` (`TextChoices: client, manager, admin`), а также поля `full_name` и `phone`. Реализован собственный `UserManager`, корректно создающий обычных пользователей и суперпользователей с автоматическим назначением роли `admin`.

Формы аутентификации. Реализованы три ключевые формы:

- `RegisterForm` (наследник `UserCreationForm`) — регистрация клиента; роль принудительно выставляется в `client`; CSS-классы Bootstrap присваиваются автоматически в `__init__`;
- `EmailAuthenticationForm` (наследник `AuthenticationForm`) — замена `username` на `EmailField`;

- ProfileForm и BootstrapPasswordChangeForm — редактирование профиля и смена пароля.

Представления. Аутентификация реализована через class-based views: RegisterView (на базе CreateView с автоматическим логином после регистрации), EmailLoginView, CustomLogoutView, ProfileView, CustomPasswordChangeView.

Ролевая модель доступа. Модуль accounts/permissions.py содержит базовый миксин RoleRequiredMixin, наследующийся от LoginRequiredMixin и UserPassesTestMixin. Базовый миксин проверяет роль пользователя и предоставляет доступ суперпользователям и пользователям с ролью admin безусловно. От него наследуются специализированные миксины ClientRequiredMixin, ManagerRequiredMixin, AdminRequiredMixin и StaffRequiredMixin (менеджер или администратор). Для функциональных view реализованы аналогичные декораторы (@manager_required, @staff_required и др.). Реализация полностью соответствует матрице прав доступа, спроектированной в подразделе 2.1.

3.3 Реализация CRUD-интерфейсов

CRUD-операции над основными сущностями реализованы в двух плоскостях: административная панель Django для технического сопровождения и публичные/менеджерские страницы на Bootstrap 5 для основной работы пользователей.

Каталог комплектующих. В catalog/views.py реализованы:

- CatalogListView — публичная страница каталога с фильтрацией (поиск по названию и бренду, выбор категории, диапазон цен, наличие на складе) и пагинацией по 12 товаров;
- ComponentDetailView — карточка товара с характеристиками из JSON-поля specs, ценой, остатком и кнопкой «В корзину»;
- ComponentManageListView (с миксином StaffRequiredMixin) — менеджерская таблица товаров с фильтрами «низкий остаток» и категориями;
- ComponentCreateView, ComponentUpdateView, ComponentDeleteView — стандартные CBV для CRUD с формой ComponentForm на Bootstrap.

Сборки ПК. В `builds/views.py` реализованы публичный список (`BuildListView`), детальная страница со составом сборки (`BuildDetailView`) и менеджерский CRUD с `inline-formset`. Класс `BuildEditView` использует `BuildItemFormSet` (`inlineformset_factory`), что позволяет редактировать состав сборки на одной странице с самой сборкой; сохранение выполняется атомарно в одной транзакции.

Заказы. В `orders/views.py` реализованы:

- `OrderListView` — история заказов клиента с пагинацией;
- `OrderDetailView` — детальная страница заказа с проверкой прав (клиент видит только свои заказы, менеджер и админ — любые);
- `OrderManageListView` — менеджерская таблица заказов с поиском и фильтрами по статусу;
- функциональное представление `manage_change_status` — смена статуса заказа менеджером.

Особого внимания заслуживает функция `_create_order`: списание остатков реализовано через выражение `F('stock')`, что гарантирует атомарность операции на уровне СУБД и полностью исключает состояние гонки (`race condition`) при одновременном оформлении нескольких заказов одного товара:

Листинг 1 – Атомарное списание остатков через `F()`-выражение

```
1 Component.objects.filter(pk=pk).update(stock=F('stock') - need)
```

Всё создание заказа обернуто в `transaction.atomic()`: при любой ошибке (нехватка остатков, ошибка записи) все изменения откатываются целиком.

Личный кабинет (дашборд). Представление `DashboardView` динамически формирует контекст в зависимости от роли пользователя: для клиента передаются последние 5 заказов с `select_related('status')` (без N+1-запросов к статусам); для менеджера и администратора — последние 10 заказов всех клиентов. Шаблон `accounts/dashboard.html` отображает: для клиента — KPI-карточку с числом заказов, ссылки на configurator и каталог,

таблицу истории заказов; для менеджера — карточки быстрого доступа к каталогу, сборкам, заказам и аналитике; для администратора — дополнительно ссылки на управление пользователями и экспорт/импорт CSV.

Каталог — улучшения UX. Страница каталога дополнена счётчиком найденных позиций в заголовке, боковой панелью быстрой фильтрации по категориям с подсветкой активной и кнопкой «Сбросить» фильтры. Пагинация реализована с сохранением GET-параметров — переход на следующую страницу не сбрасывает выбранные фильтры (все параметры передаются через цикл в URL пагинационных ссылок).

3.4 Реализация модуля сборки ПК из комплектующих (конфигуратор)

Модуль конфигуратора (URL `/builds/configure/`) реализован как одностраничное приложение на Alpine.js поверх Django-шаблона, что позволяет получить полностью реактивный пользовательский интерфейс без отдельной сборки фронтенда. Архитектурное решение — frontend получает все доступные комплектующие в момент загрузки страницы (через `json_script`) и далее работает без обращений к серверу, кроме финального сохранения сборки.

Архитектура шагов. В `builds/views.py` определена константа `CONFIGURATOR_STEPS` — список восьми шагов: процессор, материнская плата, оперативная память, накопитель, видеокарта, блок питания, корпус, охлаждение. Каждый шаг имеет поля `key`, `slug` категории, `title`, имя иконки Bootstrap Icons и флаг `required`. В контекст шаблона передаётся уже сгруппированный по шагам JSON со списком всех активных комплектующих с положительным остатком.

Серверная часть конфигуратора состоит из двух представлений:

- `ConfiguratorView` (`TemplateView`) — отдаёт страницу с шагами и JSON-данными комплектующих, отсортированных по цене;
- функциональное представление `configurator_save` (`POST`, `@login_required`) — принимает JSON со словарём `selection` (ключ шага → `id` комплектующего), валидирует, создаёт `Build` и `BuildItem`’ы атомарно (`transaction.atomic`) и опционально добавляет сборку в корзину пользователя.

Клиентская часть реализована на Alpine.js (подключён через CDN,

~15 КБ) с описанной далее логикой.

Реактивные компоненты:

- `steps` — массив шагов с комплектующими, инициализируется из `json_script`;
- `currentStep` — индекс текущего шага мастера;
- `selection` — словарь выбранных комплектующих (синхронизируется с `localStorage`);
- вычисляемые методы `filteredComponents()`, `isCompatible()`, `estimatedPower()`, `totalPrice()`, `canCheckout()`.

Правила совместимости реализованы в методе `isCompatible(component)` и проверяются динамически при отрисовке каждой карточки:

1. **Процессор ↔ материнская плата:** сокеты должны совпадать (`cpu.specs.socket === mb.specs.socket`).
2. **Материнская плата ↔ память:** тип памяти должен совпадать (`mb.specs.memory.type === ram.specs.type`; например, DDR5).
3. **Блок питания:** расчётное энергопотребление (сумма TDP процессора, TDP видеокарты и константы 80 Вт на остальные узлы) не должно превышать мощность БП. При превышении карточка БП помечается как несовместимая, при разности менее 100 Вт выводится предупреждение «БП на пределе».
4. **Кулер ↔ процессор:** расчётный TDP кулера должен покрывать TDP процессора.

Несовместимые позиции автоматически помечаются красным бейджем «несовместимо», приглушаются (`opacity: 0.5`) и блокируются от клика. На самой дешёвой совместимой позиции каждого шага автоматически отображается бейдж «рекомендуем».

Визуальные элементы интерфейса:

- **Пошаговый стайплер** с горизонтальной прокруткой и цветовой индикацией состояния (пустой, активный, выполненный);

- **Градиентный прогресс-бар** (индиго → фиолетовый → розовый), показывающий долю выполненных шагов;
- **Реактивная сводка справа** (`position: sticky`) с иконками шагов, выбранными комплектующими и ценами;
- **Индикатор мощности БП** с цветной полосой (зелёный → жёлтый → красный) и текстовой подсказкой о требуемой мощности;
- **Алерты совместимости** в верхней части страницы при конфликтах между выбранными элементами;
- **Сохранение в localStorage** — при возврате на страницу пользователь видит свой предыдущий выбор.

Завершение сборки. После выбора всех обязательных шагов разблокируются кнопки «В корзину» (с указанной суммой) и «Сохранить сборку». При нажатии выполняется `fetch`-запрос на `/builds/configure/save/` с CSRF-токеном из `cookie`; сервер создаёт сборку и возвращает URL для редиректа (страница сборки или корзина).

3.5 Реализация агрегирования данных по остаткам, продажам и прибыли

Аналитический модуль реализован в виде представления `AnalyticsView` (`accounts/views.py`) с использованием миксина `StaffRequiredMixin` (доступ только менеджерам и администраторам). Все расчёты выполняются на стороне СУБД через агрегирующие методы Django ORM, что обеспечивает приемлемую производительность даже при больших объёмах заказов.

Период выборки задаётся параметром `?days=N` (по умолчанию 30 дней; поддерживаются значения 7, 30, 90 и 365). Все выборки фильтруются по `created_at`, отменённые заказы (статус `cancelled`) исключаются.

Ключевые показатели рассчитываются за один запрос с использованием выражений `Sum` и `F`:

- **Выручка:** `Sum(F('unit_price') * F('quantity'))`;
- **Себестоимость:** `Sum(F('component_purchase_price') * F('quantity'))`;
- **Прибыль:** разность выручки и себестоимости;

- **Количество заказов и средний чек:** `Count('order', distinct=True)` и отношение выручки к количеству заказов.

Топ-10 товаров рассчитывается группировкой позиций заказов по `component_title` и `component_brand` с агрегацией `Sum('quantity')` и сортировкой по убыванию. Дополнительно выгружаются позиции с критически низкими остатками и подсчитывается количество товаров «нет в наличии».

Визуализация построена на цветных KPI-карточках (с градиентным фоном для основных показателей), таблице топ-товаров и боковом списке низких остатков с цветными бейджами (жёлтый — мало, красный — нет в наличии).

3.6 Реализация загрузки и выгрузки файлов

Модуль импорта-экспорта реализован для каталога комплектующих и отчётности по заказам.

Экспорт каталога (`ComponentExportView`) — генерирует CSV-файл с разделителем «;», кодировкой UTF-8 и предваряющим BOM (для корректного открытия в Microsoft Excel). Заголовок: `id`, `category`, `title`, `brand`, `model`, `specs`, `purchase_price`, `sale_price`, `stock`, `is_active`. Содержимое поля `specs` (JSON) сериализуется в текстовый JSON через `json.dumps(ensure`

Импорт каталога (`ComponentImportView`) — загрузка CSV-файла через стандартный `FileField`. Алгоритм импорта:

1. декодирование содержимого как `utf-8-sig` (корректная обработка BOM из Excel);
2. чтение через `csv.DictReader` с разделителем «;»;
3. для каждой строки — валидация обязательных полей (категория, название);
4. поиск или автоматическое создание категории по `slug`;
5. парсинг JSON-поля `specs`;
6. операция `Component.objects.update_or_create(title=...)`, что обеспечивает идемпотентность повторных загрузок.

По завершении импорта пользователю выводится сообщение со статистикой (создано, обновлено, пропущено) и до 10 ошибок валидации в виде flash-сообщений.

Экспорт заказов (OrderExportView) — CSV-выгрузка заказов за период с заголовком `order_id`, `date`, `client_email`, `status`, `total_sum`, `payment_method`, `delivery_method`. Период фильтруется параметрами `?from=YYYY`.

Выводы по главе 3

В главе описана программная реализация всех шести задач раздела 3 пояснительной записки. Реализованы базовая структура приложения и шаблоны страниц на Bootstrap 5 с современным дизайн-кодом, аутентификация по email и ролевая модель доступа, полноценные CRUD-интерфейсы для всех основных сущностей системы, интерактивный конфигуратор сборки ПК с проверкой совместимости в реальном времени и расчётом потребляемой мощности, аналитический модуль с расчётом выручки, прибыли, среднего чека и топ-товаров, а также модуль импорта и экспорта файлов в формате CSV. Полученная реализация полностью покрывает функциональные требования, сформулированные в подразделе 1.4.

ГЛАВА 4. ОФОРМЛЕНИЕ ИТОГОВ РАБОТЫ

В четвёртой главе описывается организация кодовой базы в системе контроля версий, тестирование и развёртывание приложения.

4.1 Организация Git-репозитория

Исходный код проекта размещён в публичном репозитории на платформе GitHub:

<https://github.com/Haskeri/kpweb>

Развёрнутое веб-приложение доступно по адресу:

<https://exam.anufriev.pvpcult.ru/>

Кодовая база проекта организована в репозитории Git. Корень репозитория содержит каталог `pcshop/` с Django-проектом, файл `requirements.txt` со списком зависимостей и файл `.gitignore`.

Репозиторий содержит полную историю разработки, сформированную в соответствии с этапами курсовой работы. История коммитов отражает последовательное выполнение разделов (таблица 4.1).

Таблица 4.1 – История коммитов репозитория

Коммит	Дата	Содержание
Раздел 1	25.04.2026	Анализ предметной области, постановка задачи, структура LaTeX-проекта
Раздел 2	16.05.2026	Проектирование архитектуры, схема БД, диаграммы компонентов
Раздел 3	06.06.2026	Разработка веб-приложения, исходный код проекта pcsshop (Django 4.2)
Раздел 4	13.06.2026	Итоговый отчёт, развёртывание, оформление пояснительной записки

Файл `.gitignore` исключает из репозитория:

- виртуальное окружение `.venv/` и кэш Python `__pycache__/`;
- базу данных `db.sqlite3` (содержит пользовательские данные);
- файл конфигурации `configs/.env` с реальными секретами;
- медиафайлы (загружаемые изображения комплектующих).

В репозитории хранится только файл `configs/.env.example` — шаблон переменных окружения без реальных значений. Это обеспечивает безопасность: секреты и чувствительные настройки не попадают в историю коммитов.

Структура файла `requirements.txt` приведена в таблице 4.2.

Таблица 4.2 – Зависимости проекта (`requirements.txt`)

Пакет	Назначение
<code>Django==4.2.16</code>	Веб-фреймворк (LTS-версия до апреля 2026 г.)
<code>Pillow==10.3.0</code>	Обработка изображений (загрузка фото комплектующих)
<code>python-decouple==3.8</code>	Чтение конфигурации из файла <code>.env</code>
<code>psycopg2-binary==2.9.9</code>	Драйвер PostgreSQL (для продакшен-развёртывания)

Установка зависимостей выполняется командой `pip install -r requirements.txt` в активированном виртуальном окружении Python 3.10+. Для инициализации базы данных и загрузки начальных данных предусмотрены команды:

Листинг 2 – Команды инициализации проекта

```
1 python manage.py migrate
2 python manage.py loaddata fixtures/order_statuses.json
3 python manage.py loaddata fixtures/categories.json
4 python manage.py loaddata fixtures/components.json
5 python manage.py createsuperuser
```

4.2 Тестирование и развёртывание приложения

Проверка системы. Встроенная команда Django `python manage.py check` выполняет статическую проверку конфигурации, URL-маршрутов, моделей и установленных приложений. Команда `python manage.py check --deploy` дополнительно проверяет настройки безопасности (HTTPS, CSRF, SECRET_KEY). Оба вида проверки не выявляют ошибок.

Ручное тестирование. Проведено функциональное тестирование всех ключевых сценариев:

- регистрация нового клиента и вход по email;
- добавление комплектующих в корзину со страницы каталога и со страницы товара;
- создание сборки в конфигураторе с проверкой совместимости;

- оформление заказа с уменьшением остатков на складе;
- смена статуса заказа менеджером;
- экспорт и импорт каталога в формате CSV.

Все сценарии выполняются без ошибок на Python 3.9 при использовании SQLite в качестве СУБД.

Развёртывание. Для локального запуска используется встроенный сервер разработки:

Листинг 3 – Запуск сервера разработки

```
1 cd pcsshop
2 source .venv/bin/activate
3 python manage.py runserver
```

Приложение становится доступным по адресу `http://localhost:8000/`. Интерактивная административная панель доступна по адресу `http://localhost:8000/admin/`.

Продакшен-развёртывание. Помимо локального запуска, проект развёрнут на хостинге и доступен по адресу:

<https://exam.anufriev.pvpcult.ru/>

На сервере используется стек: **Nginx** (обратный прокси) → **Gunicorn** (WSGI-сервер) → **Django**. Для корректной работы CSRF-защиты за HTTPS-прокси в настройках указан доверенный источник:

Листинг 4 – Ключевые параметры .env на сервере

```
1 DEBUG=False
2 ALLOWED_HOSTS=exam.anufriev.pvpcult.ru
3 CSRF_TRUSTED_ORIGINS=https://exam.anufriev.pvpcult.ru
4 SECRET_KEYдлинный=< случайный ключ>
```

Переход на продакшен-окружение предполагает замену SQLite на PostgreSQL через переменные окружения в `.env`:

Листинг 5 – Пример файла .env для PostgreSQL

```
1 SECRET_KEY=your-long-random-key
2 DEBUG=False
3 ALLOWED_HOSTS=yourdomain.com
```



```
4 DB_ENGINE=django.db.backends.postgresql
5 DB_NAME=pcshop
6 DB_USER=pcshop_user
7 DB_PASSWORD=secure_password
8 DB_HOST=localhost
9 DB_PORT=5432
```

Такой подход обеспечивает принцип «12-factor app»: конфигурация полностью отделена от кода и не попадает в репозиторий.

Выводы по главе 4

В четвёртой главе описана организация кодовой базы в системе Git с корректным `.gitignore`, разделением конфигурации через `.env` и фикстурами для первоначального заполнения базы данных. Исходный код размещён в публичном репозитории по адресу <https://github.com/Haskeri/kpweb>. Проведено ручное функциональное тестирование всех ключевых пользовательских сценариев. Приложение развёрнуто на хостинге и доступно по адресу <https://exam.anufriev.pvpcult.ru/>. Описаны команды запуска в режиме разработки и инструкция по переходу на продакшен-СУБД PostgreSQL без изменения кода приложения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было спроектировано и реализовано веб-приложение для учёта комплектующих, сборки и продажи персональных компьютеров. Поставленная цель достигнута полностью: создан готовый к внедрению программный продукт, покрывающий бизнес-процессы малой компании по продаже комплектующих и сборке ПК под заказ.

В первой главе проведён анализ предметной области, выполнен сравнительный обзор существующих программных продуктов (DNS, «Регард», «НИКС», «Ситилинк») и универсальных CRM/ERP-систем. Установлено, что распространённые системы ориентированы преимущественно на витрину товаров и не покрывают в комплексе задачи внутреннего учёта комплектующих, конфигурирования сборок и аналитики для малого бизнеса. Анализ программных инструментов разработки позволил обоснованно выбрать технологический стек на основе Python, Django, Bootstrap и PostgreSQL.

Во второй главе выполнено проектирование разрабатываемого веб-приложения: проведён анализ целевой аудитории и трёх пользовательских ролей (администратор, менеджер, клиент), построена матрица прав доступа CRUD, описаны варианты использования и оформлены 16 пользовательских историй, спроектирована модель данных на восемь сущностей с описанием связей и ограничений целостности, разработаны wireframe-макеты основных страниц приложения.

В третьей главе выполнена программная реализация. Разработана модульная структура Django-проекта pcshop из четырёх приложений, реализованы все шесть задач раздела 3:

1. базовая структура приложения и шаблоны страниц на Bootstrap 5 с современным дизайн-кодом (Inter, градиентные элементы, sticky-навбар, palette #4F46E5/#F59E0B);
2. аутентификация по email и ролевая модель доступа с миксинами и декораторами;
3. полноценные CRUD-интерфейсы для каталога, сборок, заказов и пользователей с пагинацией, фильтрами, поиском и inline-formset для сборок;
4. инновационный интерактивный конфигуратор сборки ПК на Alpine.js

с пошаговым мастером из восьми шагов, проверкой совместимости комплектующих (4 правила: сокет CPU–MB, тип памяти MB–RAM, мощность БП, TDP кулера), расчётом потребляемой мощности и сохранением состояния в `localStorage`;

5. аналитический модуль с агрегированием данных через выражения `Sum` и `F`: выручка, прибыль, средний чек, топ-10 товаров и контроль остатков с цветовой индикацией;
6. модуль импорта и экспорта файлов в формате CSV для каталога и отчётности по заказам с поддержкой BOM для корректной работы с Microsoft Excel.

Практическая значимость работы заключается в создании готового к внедрению программного продукта, который может быть использован действующими магазинами комплектующих и небольшими сборщиками ПК для автоматизации ключевых бизнес-процессов и снижения операционных издержек. Архитектура приложения позволяет масштабироваться по мере роста бизнеса (поддерживается переключение на PostgreSQL без изменения кода через переменные окружения).

Особенностью реализации является конфигуратор сборки ПК, который в отличие от рассмотренных в первой главе аналогов (DNS, НИКС, «Регард») сочетает в себе:

- реактивную проверку совместимости в момент выбора каждого компонента (без перезагрузки страницы);
- автоматическую рекомендацию оптимальной по цене конфигурации;
- расчёт энергопотребления с цветовой индикацией запаса по мощности;
- возможность одновременно положить готовую сборку в корзину как единый товарный позицию;
- сохранение прогресса в `localStorage` браузера для возврата к сборке в любой момент.

Перспективы развития. Дальнейшее развитие проекта предполагает:

- расширение правил совместимости комплектующих (поддержка стандартов PCI Express, охлаждения корпуса, форм-факторов материнских плат);
- интеграцию платёжных систем (СБП, банковский эквайринг) и системы email-уведомлений на этапах оформления и смены статуса заказа;
- подключение картографических сервисов для расчёта стоимости и сроков доставки;
- внедрение системы предиктивной аналитики для прогнозирования остатков и спроса;
- развёртывание приложения в облачной инфраструктуре с использованием Docker и Nginx.

При разработке пояснительной записки к курсовой работе строго соблюдались требования [5], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

- [1] Хокинс, М. Django для начинающих: создание веб-приложений на Python [Текст] / М. Хокинс. — М. : ДМК Пресс, 2022. — С. 320.
- [2] Cohn, M. User Stories Applied: For Agile Software Development [Text] / M. Cohn. — [S. l.] : Addison-Wesley, 2004. — P. 304.
- [3] Марков, А. С. Проектирование баз данных и информационных систем [Текст] / А. С. Марков, В. В. Новиков. — СПб. : Питер, 2021. — С. 384.
- [4] Django Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://docs.djangoproject.com/> (online; accessed: 06.04.2026).
- [5] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.

ПРИЛОЖЕНИЕ А. ДИАГРАММА ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ



Рисунок А.1 – Диаграмма вариантов использования системы pcshop (UML Use Case)